PPO, GRPO & DAPO

Policy Optimization from Trust Regions to Frontier Reasoning RL

1

Learning Objectives

- Explain why **unconstrained policy gradient updates** cause a distribution-shift death spiral
- Derive the **importance-sampled surrogate objective** that enables data reuse
- Derive and interpret the **PPO clipped surrogate objective** — all four cases and their gradients
- Trace the complete **PPO training loop**: rollout → GAE → advantage normalization → multi-epoch minibatch optimization → KL monitoring
- Map PPO onto **token-level RLHF** for LLMs (four models, per-token ratios, KL penalty)
- Derive **GRPO**: how group-relative advantages eliminate the value model entirely
- Understand **DAPO**: the four engineering fixes (Clip-Higher, Dynamic Sampling, Token-level loss, Overlong Shaping) that make GRPO work at scale
- See PPO, GRPO, RLOO, and DAPO as **variations of one principle**: policy gradient + variance-reducing baseline + trust region

From Last Time

- The policy gradient: $\nabla_\theta J = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t]$
- The **advantage** $\hat{A}_t$: this action was better or worse than expected, by how much
- **Actor-Critic + GAE** gives low-variance advantage estimates: $\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$, where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
- **Problem 1**: the gradient gives a **direction** but not a safe **step size** — how far can we move?
- **Problem 2**: this is **on-policy** — after one gradient step, all collected trajectories are stale

3

Part 1: The Stability Crisis

4

Why Large Updates Kill RL

- On-policy RL has a fragile feedback loop: **the policy generates the data that trains the policy**
- A large update changes the policy → it visits **unfamiliar states** → generates poor trajectories
- Poor trajectories produce **misleading gradients** (the landscape has shifted under you)
- The next update makes things **worse** → the policy **spirals into collapse**
- This is not hypothetical — it happens routinely in practice without safeguards
- **For LLMs**: pretrained knowledge encoded across billions of parameters can be **destroyed** by a single bad RL update. Recovery requires restarting from checkpoint.

5

The Trust Region Idea

- A learning rate constrains movement in **parameter space** — but small parameter change $\neq$ small behavior change
- Key insight: constrain how much the **policy's behavior** changes, not just the parameters
- **TRPO** (Schulman, 2015): KL divergence constraint → theoretically elegant, but requires Fisher information matrix — expensive
- **PPO** (Schulman, 2017): achieve the same trust-region effect with a **clipped first-order objective**
- No second-order math. No constrained optimization. Just a `min` and a `clamp`.
- Still the most widely used RL algorithm in production LLM alignment

6

Part 2: The PPO Clipped Objective

7

From Policy Gradient to Surrogate Objective

- The policy gradient requires **on-policy** data: $\nabla J = \mathbb{E}_{\pi_\theta}[\nabla \log \pi_\theta(a|s)\hat{A}]$
- After one gradient step, θ changes → all collected data is from the **wrong distribution**
- **Importance sampling** lets us reuse data collected under $\pi_{\theta_{\text{old}}}$:
- $$\mathbb{E}_{\pi_\theta}[f(a)] = \mathbb{E}_{\pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} f(a)\right]$$
- This gives the **surrogate objective** — evaluable on old data:
- $$L^{\text{CPI}}(\theta) = \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\hat{A}_t\right] = \hat{\mathbb{E}}_t\left[r_t(\theta)\,\hat{A}_t\right]$$
- Without constraint, maximizing L^{CPI} lets $r_t \to \infty$ — catastrophic overshoot. **PPO clips r_t directly.**

8

The Probability Ratio

- $$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$
- $r_t = 1$: action has **same probability** under old and new policies
- $r_t > 1$: action became **more likely** under the new policy
- $r_t < 1$: action became **less likely** under the new policy
- At the start of each optimization phase, $r_t = 1$ for all actions. As we take gradient steps, r_t drifts from 1.
- **PPO's job: limit how far r_t can drift from 1.**

9

The Clipped Surrogate Objective

- $$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta),\,1{-}\epsilon,\,1{+}\epsilon)\,\hat{A}_t\Big)\Big]$$
- Typical $\epsilon = 0.1$ to 0.2: the ratio is clamped to $[0.8,\ 1.2]$
- The $\min$ always selects the **more pessimistic** (conservative) estimate
- When $r_t \in [1{-}\epsilon,\ 1{+}\epsilon]$: clipped and unclipped are **identical** — normal gradient
- When r_t exceeds bounds in the beneficial direction: gradient becomes **zero** — no further push
- When r_t moves the wrong direction: gradient is **unrestricted** — free to correct mistakes

10

Interactive Demo: PPO Clipping

🚀 **Interactive Demo:** ppo_clipping_demo.html

11

The Four Cases of PPO Clipping

Advantage $\hat{A}_t$	Ratio r_t Direction	What Happens	Gradient
$\hat{A}_t > 0$ (good action)	$r_t \uparrow$ (more likely)	Encouraged — but **clips at** $1+\epsilon$	Zero beyond $1+\epsilon$
$\hat{A}_t > 0$ (good action)	$r_t \downarrow$ (less likely)	Wrong direction — gradient corrects **without limit**	Full (unclipped)
$\hat{A}_t < 0$ (bad action)	$r_t \downarrow$ (less likely)	Encouraged — but **clips at** $1-\epsilon$	Zero beyond $1-\epsilon$
$\hat{A}_t < 0$ (bad action)	$r_t \uparrow$ (more likely)	Wrong direction — gradient corrects **without limit**	Full (unclipped)

The min selects the pessimistic bound. Beneficial overshooting is prevented; wrong-direction moves are always corrected.

12

Important Subtlety: PPO Is Only an Approximate Trust Region

- Clipping creates **flat regions** in the objective beyond the clip boundary — gradient is zero there
- But clipping is **not** a hard KL constraint — the average KL can still grow too large
- Real implementations add **KL monitoring**: $\hat{D}_{\mathrm{KL}} \approx \mathbb{E}[\log \pi_{\mathrm{old}} - \log \pi_\theta]$
- Common safeguard: **early-stop PPO epochs** when KL exceeds a threshold
- Key diagnostics: **clip fraction** (0.05–0.20 is healthy), **entropy** (should decrease slowly), **value loss** (should decrease)
- In PyTorch: `ratio = torch.exp(log_prob_new - log_prob_old)`, then `clipped = torch.clamp(ratio, 1-eps, 1+eps)`

13

Part 3: The Full PPO Algorithm

14

The Complete PPO Loss

- $$\text{loss} = -\underbrace{L^{\text{CLIP}}(\theta)}_{\text{clipped surrogate}} + c_v\underbrace{(V_\phi(s_t) - G_t)^2}_{\text{value function MSE}} - c_{\text{ent}}\underbrace{H[\pi_\theta(\cdot|s_t)]}_{\text{entropy bonus}}$$
- **Clipped surrogate**: makes good actions more likely, bad actions less likely — within the trust region
- **Value loss**: trains the critic V_ϕ to predict returns. Target: $G_t = \hat{A}_t + V_{\phi_{\text{old}}}(s_t)$
- **Entropy bonus**: prevents premature convergence, encourages exploration
- Typical: $c_v = 0.5$, $c_{\text{ent}} = 0.01$, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, $K = 3$-5 epochs

15

PPO Training Loop

```
Initialize policy π_θ, value function V_φ
Set θ_old ← θ

for iteration = 1, 2, ... do

  ┌─ STEP 1: ROLLOUT ──────────────────────────────────────┐
  │ Freeze current policy as π_old                         │
  │ Collect trajectories using π_old                       │
  │ Store: s_t, a_t, r_t, log π_old(a_t|s_t), V_φ(s_t)  │
  └────────────────────────────────────────────────────────┘

  ┌─ STEP 2: COMPUTE ADVANTAGES (GAE) ────────────────────┐
  │ for t = T-1 down to 0:                                 │
  │   δ_t = r_t + γ·V_φ(s_{t+1}) - V_φ(s_t)            │
  │   Â_t = δ_t + (γλ)·Â_{t+1}                            │
  │ G_t = Â_t + V_φ(s_t)            // return targets     │
  │ Normalize: Â ← (Â - μ) / (σ + ε)                      │
  └────────────────────────────────────────────────────────┘

  ┌─ STEP 3: OPTIMIZE (K epochs over minibatches) ────────┐
  │ for epoch = 1 to K:                                    │
  │   Shuffle buffer into minibatches                      │
  │   for each minibatch:                                  │
  │     r_t(θ) = exp(log π_θ(a_t|s_t) - log π_old(...)) │
  │     L_clip = min(r_t·Â_t, clip(r_t,1-ε,1+ε)·Â_t)   │
  │     L_value = (V_φ(s_t) - G_t)²                        │
  │     L_entropy = H[π_θ(·|s_t)]                          │
  │     loss = -mean(L_clip) + c_v·mean(L_value)           │
  │            - c_ent·mean(L_entropy)                     │
  │     gradient_step(θ, φ)                                │
  │                                                        │
  │   approx_kl = mean(log π_old - log π_θ)               │
  │   if approx_kl > kl_target: early-stop epochs         │
  └────────────────────────────────────────────────────────┘

  θ_old ← θ, collect fresh trajectories
```

Implementation Details That Matter

- **Store old log-probs** at rollout time — do not recompute them (they must come from π_{old})
- **Normalize advantages** within the batch: $\hat{A} \leftarrow (\hat{A} - \mu)/(\sigma + \epsilon)$ — critical for stability
- **Multi-epoch reuse**: PPO safely reuses the same batch for $K = 3\text{-}10$ epochs — $K\times$ sample efficiency over REINFORCE
- **Compute ratios in log space**: `ratio = exp(log_prob_new - log_prob_old)` to avoid numerical issues
- **Gradient clipping**: $\|\nabla\|_{\max} = 1.0$ for additional stability
- **Mask terminal states, padding, and invalid actions** correctly

17

Part 4: PPO for Language Models (RLHF)

18

The LLM RL Setup: Four Models in Memory

- **Actor** (policy π_θ): the transformer — $\pi_\theta(y_t|x, y_{<t})$
- **Critic** (value model V_ϕ): same backbone, scalar head — $V_\phi(x, y_{\leq t}) \in \mathbb{R}$
- **Reference model** (π_{ref}): frozen copy of the SFT model — anchors the policy
- **Reward model** (R_ψ): scores complete responses — $R_\psi(x, y) \in \mathbb{R}$
- For a 70B model: **four copies** ≈ 280B parameters in GPU memory
- Per-token ratio: $r_t(\theta) = \pi_\theta(y_t \mid x, y_{<t}) \,/\, \pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})$
- **KL penalty** to reference: $r'_t = r_t - \beta \log \frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\text{ref}}(y_t|x,y_{<t})}$ — prevents reward hacking

19

Interactive Demo: PPO Training Loop

🚀 **Interactive Demo:** ppo_training_demo.html

20

Part 5: GRPO — Eliminating the Value Model

21

The Problem with PPO's Critic

- PPO's value model is **another full LLM** — doubles model memory beyond actor + reference + reward
- Training an accurate value function for LLM generation is **hard**: the 'state' is the entire context, rewards are sparse
- A noisy critic → noisy advantages → unstable training. The critic becomes the **bottleneck**.
- **DeepSeek's insight** (GRPO, 2024): generate **multiple responses** per prompt, use them as **mutual baselines**
- No learned value function needed → **3 models instead of 4**

22

Group Relative Advantages

- For each prompt q, sample a **group** of G responses: $\{o_1, \ldots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot|q)$
- Score each with a reward model or verifier: $\{r_1, \ldots, r_G\}$
- Compute advantages by **normalizing within the group**:
- $$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$
- This is **REINFORCE with the group mean as baseline** — apples-to-apples comparison
- Key difference from PPO: $\hat{A}_i$ is a **single scalar per response** — all tokens share the same advantage
- Closely related to **RLOO** (Leave-One-Out baseline): same family, slightly different baseline computation

23

The GRPO Objective

- $\mathcal{J}_{\text{GRPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}\,\hat{A}_i,\ \text{clip}(r_{i,t},\ 1{-}\epsilon,\ 1{+}\epsilon)\,\hat{A}_i\Big) - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})$
- **Same PPO clipping mechanism** — just a different way of computing advantages
- Per-token ratio: $r_{i,t}(\theta) = \pi_\theta(o_{i,t}|q, o_{i,<t}) \,/\, \pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})$
- **KL penalty** directly in the loss (not in the reward), keeping the policy near π_{ref}
- $1/|o_i|$ normalizes by response length so every response contributes equally
- **Used by**: DeepSeek-R1, DeepSeek-V3, Qwen 3, GLM-5 — the dominant algorithm for reasoning RL

24

Interactive Demo: GRPO

🚀 **Interactive Demo:** grpo_demo.html

25

GRPO Training Loop

```
Initialize policy π_θ, reference policy π_ref ← π_θ
Set θ_old ← θ
NO VALUE MODEL NEEDED

for iteration = 1, 2, ... do

   ┌─ STEP 1: SAMPLE GROUPS ──────────────────────────────┐
   │ Sample prompts {q_1, ..., q_B}                        │
   │ For each prompt q_j:                                  │
   │   Generate G responses: {o_1,...,o_G} ~ π_θ_old(·|q)  │
   │   Score each: {r_1,...,r_G} via reward/verifier       │
   │   Store: responses, rewards, log π_θ_old(o_{i,t}|.)  │
   └───────────────────────────────────────────────────────┘

   ┌─ STEP 2: COMPUTE GROUP ADVANTAGES ──────────────────┐
   │ For each prompt q_j:                                 │
   │   μ = mean(r_1,...,r_G),  σ = std(r_1,...,r_G)       │
   │   Â_i = (r_i - μ) / σ   for each response            │
   │   (One scalar per response — all tokens share it)    │
   └──────────────────────────────────────────────────────┘

   ┌─ STEP 3: PPO-STYLE UPDATE (K epochs) ───────────────┐
   │ for epoch = 1 to K:                                  │
   │   for each minibatch:                                │
   │     r_{i,t} = π_θ(o_{i,t}|q,o_{i,<t})                │
   │             / π_θ_old(o_{i,t}|q,o_{i,<t})            │
   │     L = min(r·Â, clip(r,1-ε,1+ε)·Â)                  │
   │     KL = approx_kl(π_θ, π_ref)                       │
   │     loss = -mean(L / |o_i|) + β·mean(KL)             │
   │     gradient_step(θ)                                 │
   └──────────────────────────────────────────────────────┘

  θ_old ← θ
```

26

Part 6: DAPO — Making GRPO Work at Scale

27

GRPO's Scaling Problems

- When teams tried to reproduce DeepSeek-R1 with vanilla GRPO, they hit serious problems
- **Entropy collapse**: the policy becomes deterministic too early — exploration dies, sampled responses become nearly identical
- **Dead groups**: if all G responses in a group get the same reward (all correct or all wrong), $\mathrm{std} = 0$ → advantage is undefined → zero gradient, wasted compute
- **Length bias**: normalizing loss by $1/|o_i|$ makes short responses get disproportionately large gradients
- **Truncation noise**: responses hitting the max length limit get cut off mid-thought → noisy reward signal
- DAPO (ByteDance, 2025) fixes all four problems — enabling GRPO to work reliably at scale

28

DAPO's Four Fixes

- **1. Clip-Higher (Decoupled Clipping)**: use asymmetric clip bounds — e.g., $\epsilon_{\text{low}} = 0.2$, $\epsilon_{\text{high}} = 0.28$. The wider upper bound lets the policy explore more (increase probability of surprising new tokens) while the tighter lower bound maintains stability
- **2. Dynamic Sampling**: skip groups where all rewards are identical ($\text{std} = 0$). Over-sample prompts and filter until the batch is full of informative groups
- **3. Token-Level Loss**: instead of $1/|o_i|$ per-response normalization, normalize across all tokens in the batch equally. Removes length bias — critical for long Chain-of-Thought
- **4. Overlong Reward Shaping**: mask truncated responses from loss entirely (they contain noise). Add a soft length penalty that ramps linearly as responses approach the max length
- **Bonus: Drop the KL penalty**: multiple groups found that for reasoning tasks with verifiable rewards, the KL term is unnecessary and can hurt by being too conservative

29

DAPO Results

- Trained Qwen2.5-32B from ~0% to **50% on AIME 2024** (competitive math)
- Outperformed DeepSeek-R1-Zero-Qwen-32B (47%) with **50% fewer training steps**
- Vanilla GRPO on the same setup: only 30% — DAPO's fixes account for **+20 points**
- Each technique contributes several points independently; together they compound
- Now the standard baseline for open-source reasoning RL — implemented in TRL and verl frameworks

30

PPO vs. GRPO vs. DAPO

	PPO	GRPO	DAPO
Value model	**Yes** — full LLM critic	**No**	**No**
Models in memory	4 (actor, critic, ref, reward)	3 (actor, ref, reward/verifier)	3 (actor, ref, reward/verifier)
Advantage	**Per-token** via GAE	**Per-response**: $(r_i - \mu)/\sigma$	Same as GRPO + fixes
KL handling	Per-token reward shaping	Loss penalty term	**Dropped** for RLVR tasks
Clipping	Symmetric ϵ	Symmetric ϵ	**Asymmetric**: $\epsilon_{\text{low}} < \epsilon_{\text{high}}$
Dead groups	N/A (single response)	Zero gradient, wasted compute	**Dynamic sampling** filters them
Length bias	N/A	Yes ($1/\lvert o_i \rvert$ normalization)	**Fixed** with token-level loss
Samples per prompt	1 response	G responses (8–64)	G responses + oversampling
Best for	General alignment (RLHF)	Reasoning RL (RLVR)	Reasoning RL at scale
Used by	OpenAI, Anthropic, Google	DeepSeek-R1, Qwen	Open-source SOTA, frontier labs

All three share the same PPO clipping mechanism. They differ in advantage computation, memory cost, and engineering robustness.

Part 7: The Unified View

32

All These Algorithms Are Variations of One Idea

- **Core principle**: policy gradient + variance-reducing baseline + trust region (clipping)
- **PPO**: baseline = learned value function $V_\phi(s_t)$ via GAE → per-token advantage
- **GRPO**: baseline = group mean reward → per-response advantage
- **RLOO**: baseline = leave-one-out mean → per-response advantage (slightly lower variance than GRPO)
- **REINFORCE++**: REINFORCE + PPO's clipping + normalization tricks (no critic, single sample)
- The **clipping logic is identical** across PPO/GRPO/DAPO/RLOO — only the advantage computation differs
- In practice, **infrastructure and data often matter more than algorithmic differences** between these variants — but the differences matter at scale

33

RLVR: The Paradigm Shift

- **RLHF** (2022–2024): human preferences → trained reward model → PPO. Four models. Expensive labels.
- **RLVR** (2024–present): verifiable rewards from code execution, math checkers, rule-based verifiers → GRPO/DAPO. **No reward model. No critic.** Just actor + reference.
- DeepSeek-R1 demonstrated that pure RLVR can produce **emergent reasoning** — self-verification, backtracking, iterative refinement — from scratch
- This is now the standard for reasoning RL: math, code, logic, tool use
- The frontier stack in 2025–2026: **SFT** (instruction following) → **Preference optimization** (DPO/alignment) → **RLVR** (reasoning capabilities)
- **Next lecture**: where the reward signal comes from — reward models, DPO, verifiers, and how these compose into multi-stage training

34

Summary

35

All Interactive Demos

🚀 **Interactive Demo:** ppo_clipping_demo.html

🚀 **Interactive Demo:** ppo_training_demo.html

🚀 **Interactive Demo:** grpo_demo.html

36

Lecture Summary

- **The problem**: unconstrained policy updates → death spiral
- **Importance sampling** enables data reuse: surrogate $L = \mathbb{E}[r_t \hat{A}_t]$ with $r_t = \pi_\theta / \pi_{\text{old}}$
- **PPO clipping**: $\min(r_t \hat{A}_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)$ — conservative, one line of code
- **Full PPO**: clipped policy + value function + entropy bonus. **4 models for LLMs.** Per-token credit assignment via GAE.
- **GRPO**: sample G responses, normalize rewards → per-response advantage. No critic. **3 models.** Same clipping.
- **DAPO**: asymmetric clipping, dynamic sampling, token-level loss, overlong shaping. GRPO that actually works at scale.
- **Unified view**: all are policy gradient + baseline + trust region. The choice is about **what baseline** and **how many models**.
- **Next lecture**: Where does the reward come from? → Reward models, DPO, verifiers

37

Supplementary Resources

🚀 **Interactive Demo:** handout.html

38